

Engineering



& Physics

PHYS 351

Topic 1:

Linear Systems

Dr. Daugherty
Abilene Christian University

Starting Simple

This semester we will progress from very simple to very hard problems.

Very easy: $x = 2$

Easiest non-trivial form: $2x = 4$

Starting Simple

Consider the form:

$$Ax = B$$

for constants A, B .

- Does the solution always work? When does it break?
- What does the broken math tell us about the problem

Linear Systems

Solving one linear equation is really easy. What should we do next?

Two options for interesting problems:

- 1) Non-linear equations
- 2) A system of linear equations to solve simultaneously

Today we'll do option 2. We'll hit option 1 next week.

Solving Diagonal Systems (1)

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

is equivalent to

$$\begin{aligned} x_1 &= -1 \\ 3x_2 &= 6 \\ 5x_3 &= -15 \end{aligned}$$

The solution is

$$x_1 = -1 \qquad x_2 = \frac{6}{3} = 2 \qquad x_3 = \frac{-15}{5} = -3$$

Statement #1 – Solving diagonal systems is easy

Solving Triangular Systems (6)

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

is equivalent to

$$\begin{array}{rclcl} -2x_1 & + & x_2 & + & 2x_3 & = & 9 \\ & & 3x_2 & + & -2x_3 & = & -1 \\ & & & & 4x_3 & = & 8 \end{array}$$

Solve in backward order (last equation is solved first)

$$x_3 = \frac{8}{4} = 2$$

$$x_2 = \frac{1}{3}(-1 + 2x_3) = \frac{3}{3} = 1$$

$$x_1 = \frac{1}{-2}(9 - x_2 - 2x_3) = \frac{4}{-2} = -2$$

Statement #2 – Solving triangular systems is also easy

Gaussian Elimination — Cartoon Version (2)

Eliminate elements under the pivot element in the first column. x' indicates a value that has been changed once.

$$\begin{bmatrix} \boxed{x} & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix} \longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix}$$

Goal: make A triangular

$$\longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ x & x & x & x & x \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \end{bmatrix}$$

Gaussian Elimination — Cartoon Version (3)

The pivot element is now the diagonal element in the second row. Eliminate elements under the pivot element in the second column. x'' indicates a value that has been changed twice.

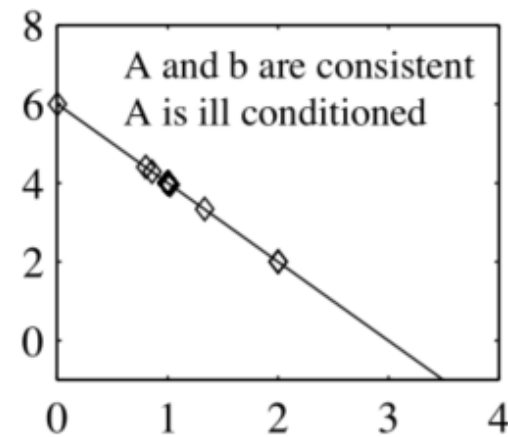
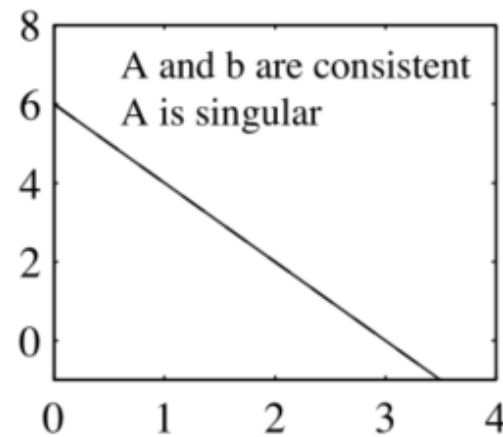
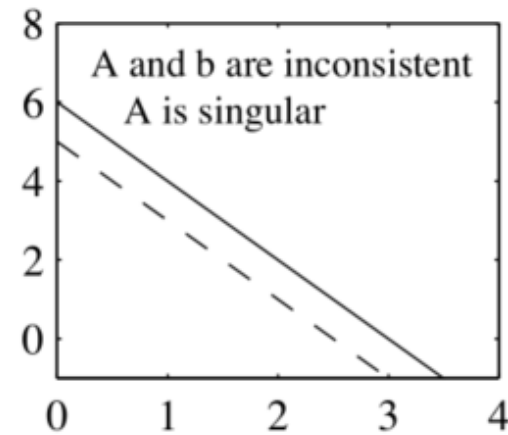
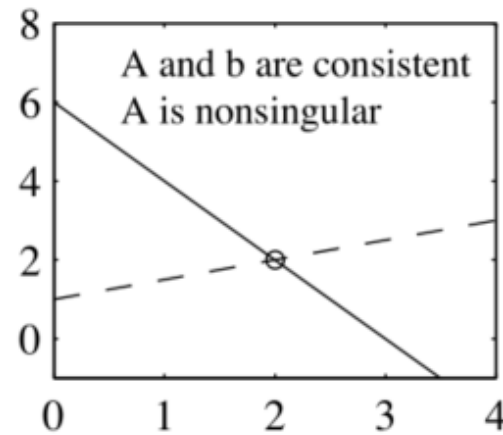
$$\begin{array}{ccc} \begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \end{bmatrix} & \longrightarrow & \begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & x' & x' & x' & x' \end{bmatrix} \\ & & \longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & 0 & x'' & x'' & x'' \end{bmatrix} \end{array}$$

Singularity of A

If an $n \times n$ matrix, A , is **singular** then

- the columns of A are linearly dependent
- the rows of A are linearly dependent
- $\text{rank}(A) < n$
- $\det(A) = 0$
- A^{-1} does not exist
- a solution to $Ax = b$ may not exist
- If a solution to $Ax = b$ exists, it is not unique

Geometric Interpretation of Singularity (3)



Tiny fluctuations in numbers
cause a big change in solution

Ill-Conditioned Example

Here we see that adjusting the coefficients by a tiny value ($1e-15$) changes the result by 5%

```
1 delta = np.logspace(-15, -10, 20)
2 for d in delta:
3     A = np.array([[2,1],[2+d,1]])
4     b = np.array([6,6+d])
5     x = np.linalg.solve(A,b)
6     print(f"{d:e}\t{x}")
7     #print(f"{d:e}\t{x}\t\t{np.linalg.det(A):0.4e}\t{np.linalg.cond(A):0.4e}")
```

delta	solution
1.000000e-15	[1. 4.]
1.832981e-15	[1. 4.]
3.359818e-15	[1. 4.]
6.158482e-15	[1. 4.]
1.128838e-14	[1. 4.]
2.069138e-14	[0.95744681 4.08510638]
3.792690e-14	[1. 4.]

You are in the danger zone if the determinant of A is small (such as $<1e-10$)



```
1 A = np.array([[ -0.5, 1], [2, 1]])
2 print(A)
3
4 b = np.array([1, 6])
5 print(b)
6
7 np.linalg.solve(A, b)
```

```
[[ -0.5   1. ]
 [  2.    1. ]]
[1  6]
array([2.,  2.])
```

numpy.linalg.solve

`linalg.solve(a, b)`

[\[source\]](#)

Solve a linear matrix equation, or system of linear scalar equations.

Computes the “exact” solution, x , of the well-determined, i.e., full rank, linear matrix equation $ax = b$.

Parameters: $a : (\dots, M, M)$ *array_like*

Coefficient matrix.

$b : \{(\dots, M,), (\dots, M, K)\}$, *array_like*

Ordinate or “dependent variable” values.

Returns: $x : \{(\dots, M,), (\dots, M, K)\}$ *ndarray*

Solution to the system $a x = b$. Returned shape is identical to b .

Raises: `LinAlgError`

If a is singular or not square.